Técnicas de EDA y preprocesado con Python

Proyecto práctico reproducible

Material didáctico

27 de octubre de 2026

Resumen

Este proyecto presenta un flujo completo de análisis exploratorio de datos (EDA) y preparación de datos para regresión. Se trabaja con un CSV sintético de expedientes de terrazas incluido en la carpeta `datos/`. El objetivo es aprender a revisar calidad, describir distribuciones y relaciones, detectar valores atípicos sin eliminarlos automáticamente y preparar particiones de entrenamiento y prueba evitando fuga de información.

Índice

1. Objetivos y alcance **2**

2. Estructura del proyecto **2**

3. Datos y pregunta de análisis **2**

4. EDA: técnicas y decisiones **2**
- 4.1. Inspección y calidad . . . 2
- 4.2. Univariado . . . 3
- 4.3. Bivariado y multivariado . . . 3
- 4.4. Valores atípicos . . . 3

5. Preprocesado **3**
- 5.1. Separación de roles . . . 3
- 5.2. Partición y fuga de información . . . 3
- 5.3. Transformaciones elegidas . . . 3

6. Ejecución **4**

7. Interpretación de resultados **4**

8. Limitaciones y buenas prácticas **4**

9. Lista de comprobación **4**

10. Referencias técnicas **4**

1. Objetivos y alcance

Al terminar, el estudiante podrá:

- inspeccionar estructura, tipos, cardinalidad, valores ausentes y duplicados;
- resumir variables numéricas y categóricas con tablas y gráficos;
- examinar relaciones y correlaciones, distinguiendo asociación de causalidad;
- localizar posibles valores atípicos y decidir su tratamiento con contexto;
- separar objetivo, identificador y predictores;
- dividir los datos antes de ajustar imputadores, escaladores y codificadores;
- guardar salidas y hacer el flujo reproducible.

El conjunto es deliberadamente sintético. `x` es un índice didáctico de complejidad y `y` son días simulados de tramitación: no representan expedientes ni resultados reales.

2. Estructura del proyecto

```
proyecto_eda_preprocesado_latex/
|-- main.tex
|-- datos/datos_ejemplo_complejo.csv
|-- scripts/eda_preprocesado.py
|-- requirements.txt
|-- README.md
|-- resultados/                 (se crea al ejecutar)
```

3. Datos y pregunta de análisis

El archivo contiene las columnas:

Columna	Interpretación didáctica
`expediente_id`	Identificador; no debe ser predictor.
`distrito`	Categoría territorial sintética.
`tipo_tramite`	Categoría del trámite.
`superficie_solicitada_m2`	Medida numérica simulada.
`requerimientos_documentacion`	Conteo discreto sintético.
`mes_presentacion`	Mes de presentación, de 1 a 12.
`x`	Índice didáctico de complejidad.
`y`	Objetivo: días simulados.

Pregunta orientativa: ¿qué patrones descriptivos aparecen entre las características de entrada y `y`? Esta exploración no demuestra causalidad ni valida conclusiones para situaciones reales.

4. EDA: técnicas y decisiones

4.1. Inspección y calidad

Comenzar con dimensiones, muestra de filas, tipos, estadísticas y porcentaje de valores ausentes. Revisar duplicados exactos y duplicidad del identificador por separado. Un duplicado puede ser un error, pero también una observación legítima; investigar antes de borrar.

```
df.shape
df.head()
df.info()
df.describe(include="all").T
```

```
df.isna().sum().sort_values(ascending=False)
df.duplicated().sum()
df["expediente_id"].duplicated().sum()
```

4.2. Univariado

Para variables numéricas usar histogramas, densidades, cuantiles y diagramas de caja. Para categóricas, contar frecuencias y usar barras. Observar asimetrías, clases raras, rangos imposibles y posibles errores de captura.

4.3. Bivariado y multivariado

Usar dispersión para pares numéricos, boxplots para comparar una variable numérica por categoría y correlación para resumir asociaciones lineales entre columnas numéricas. La correlación de Pearson puede ser baja aunque exista una relación no lineal y no mide causalidad. Para categorías, comparar proporciones y tamaños de grupo antes de interpretar.

4.4. Valores atípicos

La regla IQR marca como candidatos los valores fuera de $[Q_1 - 1{,}5\,IQR,\ Q_3 + 1{,}5\,IQR]$. Es una señal para investigar, no una orden de eliminar. Confirmar unidades, reglas de negocio, errores y sensibilidad del modelo. En este proyecto se conservan y se visualizan.

5. Preprocesado

5.1. Separación de roles

`expediente_id` se conserva como metadato, pero se excluye de `X`. `y` es el objetivo. El resto de campos constituyen predictores. No se deben incluir columnas derivadas del objetivo ni información que solo estaría disponible después del evento que se pretende anticipar.

5.2. Partición y fuga de información

Primero se crea una partición train/test. Después se ajustan las transformaciones únicamente con train. Ajustar una media, una mediana o un escalador sobre todos los datos deja que test influya en el entrenamiento y produce una evaluación demasiado optimista. Un `Pipeline` y un `ColumnTransformer` ayudan a mantener este orden también durante validación cruzada.

5.3. Transformaciones elegidas

Tipo	Transformación	Motivo
Numérica	Imputar mediana	Robusta frente a valores extremos.
Numérica	Estandarizar	Pone escalas comparables; útil para modelos sensibles a escala.
Categórica	Imputar moda	Completa categorías ausentes.
Categórica	One-hot	Representa categorías sin imponer orden artificial.

Para árboles, el escalado suele ser innecesario; para SVR, KNN y regularización suele ser importante. Si existen categorías ordinales reales, codificarlas con criterio explícito. Para alta cardinalidad considerar agrupación justificada u otros codificadores dentro de validación.

6. Ejecución

Instalar Python 3.10 o superior y ejecutar desde la raíz del proyecto:

```
python -m venv .venv
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
python -m pip install -r requirements.txt
python scripts/eda_preprocesado.py
```

Las salidas se escriben en `resultados/`. Para cambiar el origen o el objetivo, revisar las constantes del script y mantener `expediente_id` excluido.

7. Interpretación de resultados

El script calcula número de filas y columnas, duplicados, faltantes, perfiles y estadísticas. Produce histogramas, boxplots, correlaciones y una gráfica de relación entre `x` e `y`. También crea train/test preprocesados y serializa el transformador. Al interpretar:

1. comprueba que la muestra y los rangos son razonables;
2. lee conteos junto con porcentajes y tamaños de grupo;
3. investiga candidatos atípicos con contexto antes de tratarlos;
4. examina relaciones gráficas además de correlaciones;
5. valida las decisiones con conocimiento del dominio;
6. documenta cambios para que otro analista pueda reproducirlos.

8. Limitaciones y buenas prácticas

El CSV se generó para enseñanza y no permite inferir patrones administrativos reales. Los gráficos descriptivos no prueban causalidad. El split aleatorio es adecuado para este ejemplo independiente; datos temporales, grupos o entidades repetidas pueden requerir particiones temporales o agrupadas. La transformación de fecha, texto libre, clases desequilibradas y valores imposibles debe adaptarse al conjunto real. El preprocesador serializado debe cargarse solo desde fuentes confiables.

9. Lista de comprobación

- ☐ ¿Se identificaron objetivo, predictores e identificadores?
- ☐ ¿Se inspeccionaron nulos, duplicados, tipos, rangos y categorías raras?
- ☐ ¿Se estudiaron distribuciones y relaciones con gráficos?
- ☐ ¿Los candidatos atípicos se investigaron en vez de borrarse automáticamente?
- ☐ ¿Se separó test antes de ajustar transformaciones?
- ☐ ¿Se guardaron decisiones, dependencias y resultados reproducibles?

10. Referencias técnicas

- Documentación de pandas: https://pandas.pydata.org/docs/

- Preprocesado de scikit-learn: https://scikit-learn.org/stable/modules/preprocessing.html
- Composición de transformaciones: https://scikit-learn.org/stable/modules/compose.html
- Validación cruzada y fuga de datos: https://scikit-learn.org/stable/common_pitfalls.html